

SharedPreferences全解析（Java）及第三方替代库详解

在Android开发中，数据持久化是基础需求之一，**SharedPreferences**（简称**SP**）是官方提供的轻量级存储方案，主要用于存储键值对形式的简单数据（如用户配置、登录状态、开关设置等）。本文将用Java语言完整讲解SP的使用，并深入介绍几款主流的第三方替代库，帮助开发者在实际项目中做出更合适的选择。

一、SharedPreferences 核心解析（官方方案）

1. 核心概念

SharedPreferences本质是基于XML文件的存储工具，数据以键值对（key-value）形式存储在 `/data/data/应用包名/shared_prefs/` 目录下的XML文件中。其特点是：

- 轻量级：适合存储少量数据（如配置信息，不建议存大量数据）；
- 线程安全：官方提供的 `edit()` 方法通过同步锁保证线程安全；
- 支持的数据类型：boolean、int、float、long、String、Set<String>;
- 默认内存缓存：读取时先从内存获取，提升效率。

2. 完整使用流程（Java）

SP的使用分为“获取实例→读写数据→提交修改”三个核心步骤，以下是完整代码示例。

（1）获取SharedPreferences实例

通过 `Context` 的两种方法获取，核心区别是存储的XML文件名不同：

Code block

```
1 // 方法1: 指定XML文件名（推荐，便于管理）
2 // 文件名: MyAppSP.xml, 模式: MODE_PRIVATE（仅当前应用可访问）
3 SharedPreferences sp = getSharedPreferences("MyAppSP", Context.MODE_PRIVATE);
4
5 // 方法2: 使用默认文件名（默认名为"包名_preferences", 不推荐，易混淆）
6 SharedPreferences defaultSp =
    PreferenceManager.getDefaultSharedPreferences(this);
```



注意：Android 7.0（API 24）后，`MODE_WORLD_READABLE` 和 `MODE_WORLD_WRITEABLE` 已废弃，避免跨应用访问带来的安全风险。

(2) 读取数据

通过SP的 `getXxx(key, 默认值)` 方法读取对应类型数据，默认值在key不存在时生效：

Code block

```
1 // 读取基本类型
2 String userName = sp.getString("user_name", "未知用户"); // key不存在时返回"未知用户"
3 int userAge = sp.getInt("user_age", 0);
4 boolean isLogin = sp.getBoolean("is_login", false);
5 float userScore = sp.getFloat("user_score", 0.0f);
6 long loginTime = sp.getLong("login_time", 0);
7
8 // 读取Set<String> (如用户标签)
9 Set<String> userTags = sp.getStringSet("user_tags", new HashSet<>());
```

(3) 写入/修改数据

写入必须通过 `Editor` 编辑器，支持两种提交方式，核心区别是“是否异步”：

Code block

```
1 // 1. 获取编辑器
2 SharedPreferences.Editor editor = sp.edit();
3
4 // 2. 写入数据 (支持链式调用)
5 editor.putString("user_name", "张三")
6     .putInt("user_age", 25)
7     .putBoolean("is_login", true)
8     .putFloat("user_score", 98.5f)
9     .putLong("login_time", System.currentTimeMillis())
10    .putStringSet("user_tags", new HashSet<>(Arrays.asList("学生",
11    "Android"))));
12
13 // 3. 提交修改 (二选一)
14 // 方式1: 同步提交 (阻塞当前线程, 返回布尔值表示是否成功, 适合重要数据)
15 boolean isSuccess = editor.commit();
16
17 // 方式2: 异步提交 (非阻塞, 无返回值, 适合非核心数据, Android 2.3+支持)
18 editor.apply();
```

(4) 删除数据

通过编辑器的 `remove(key)` 删除指定键，或 `clear()` 清空所有数据：

Code block

```
1  SharedPreferences.Editor editor = sp.edit();
2  editor.remove("user_score"); // 删除单个键
3  // editor.clear(); // 清空所有数据
4  editor.apply();
```

(5) 监听数据变化

通过 `registerOnSharedPreferenceChangeListener` 监听SP数据变化，适合跨页面同步配置：

Code block

```
1  // 注册监听器
2  sp.registerOnSharedPreferenceChangeListener(new
    SharedPreferences.OnSharedPreferenceChangeListener() {
3      @Override
4      public void onSharedPreferenceChanged(SharedPreferences sharedPreferences,
        String key) {
5          // 当key对应的数据变化时回调
6          if ("is_login".equals(key)) {
7              boolean newLoginState = sharedPreferences.getBoolean(key, false);
8              // 执行登录状态变化后的逻辑（如更新UI）
9              updateLoginUI(newLoginState);
10         }
11     }
12 });
13
14 // 注意：页面销毁时建议解注册，避免内存泄漏
15 @Override
16 protected void onDestroy() {
17     super.onDestroy();
18     sp.unregisterOnSharedPreferenceChangeListener(listener); // listener为上面的
    实例
19 }
```

3. SP的优缺点及适用场景

(1) 优点

- 官方原生：无需引入第三方依赖，兼容性好；

- 使用简单：API简洁，上手成本低；
- 线程安全：Editor的commit/apply方法保证同步安全。

(2) 缺点

- 数据类型有限：仅支持6种基础类型，无法直接存储对象/集合（需手动序列化）；
- 大量数据性能差：XML文件存储，读写大量数据时IO效率低；
- 不支持复杂查询：无查询条件，只能通过key精确获取；
- apply()潜在风险：异步提交在应用崩溃时可能丢失数据；
- 代码冗余：频繁读写时，重复的put/get代码繁琐。

(3) 适用场景

适合存储“少量、简单、键值对”数据，如：

- 用户登录状态（isLogin）；
- 应用主题设置（如深色模式开关）；
- 界面配置（如字体大小、是否显示引导页）；
- 临时缓存的简单数据（如上次登录时间）。

二、SharedPreferences 第三方替代库详解

针对SP的缺点，第三方库通过“支持对象存储、优化IO性能、简化API”等方式进行增强。以下是4款主流替代库的详细对比及Java使用教程，涵盖使用频率最高的 `MMKV`、`DataStore` 等。

1. MMKV —— 腾讯出品，性能之王（推荐）

MMKV是腾讯开源的基于 `mmap` 内存映射的键值对存储库，解决了SP的IO性能问题，支持更多数据类型，是目前Android开发中SP的首选替代品。

(1) 核心特性

- 性能极强：基于mmap，读写速度比SP快10-100倍；
- 支持多种数据类型：除SP的基础类型外，还支持对象、集合（无需序列化）；
- 线程安全：自带锁机制，支持多线程并发访问；
- 跨进程支持：可配置跨进程访问（需开启相关设置）；
- 兼容性好：支持Android 4.0（API 14）及以上版本。

(2) 集成步骤

在模块级 `build.gradle` 中添加依赖，同步后即可使用：

Code block

```
1 // 模块级 build.gradle (Module: app)
2 dependencies {
3     // MMKV 核心依赖 (最新版本可从GitHub获取)
4     implementation 'com.tencent:mmkv:1.3.1'
5 }
```

(3) 完整使用流程 (Java)

MMKV的API比SP更简洁，无需手动获取Editor，支持直接读写，核心步骤为“初始化→读写数据”。

① 初始化 (全局仅需一次)

建议在Application类中初始化，确保全局可用：

Code block

```
1 public class MyApp extends Application {
2     @Override
3     public void onCreate() {
4         super.onCreate();
5         // 初始化MMKV (默认存储路径: /data/data/应用包名/files/mmkv/)
6         MMKV.initialize(this);
7
8         // 可选: 自定义存储路径 (如SD卡, 需申请权限)
9         // String rootDir =
10         Environment.getExternalStorageDirectory().getAbsolutePath() + "/MyApp/MMKV";
11         // MMKV.initialize(this, rootDir);
12     }
13 }
```

记得在 `AndroidManifest.xml` 中配置Application：

Code block

```
1 <application
2     android:name=".MyApp"
3     ...>
4 </application>
```

② 获取MMKV实例

支持多实例（对应不同存储文件），通过ID区分，默认实例可直接通过 `MMKV.defaultMMKV()` 获取：

```

1 // 方法1: 默认实例 (对应默认存储文件)
2 MMKV mmkv = MMKV.defaultMMKV();
3
4 // 方法2: 指定ID (对应独立存储文件, 便于分类管理, 如"UserInfo"存储用户信息)
5 MMKV userMMKV = MMKV.mmkvWithID("UserInfo");
6
7 // 方法3: 开启跨进程访问 (需指定mode为MMKV.MULTI_PROCESS_MODE)
8 MMKV multiProcessMMKV = MMKV.mmkvWithID("CrossProcess",
    MMKV.MULTI_PROCESS_MODE);

```

③ 读写数据 (支持更多类型)

MMKV支持SP的所有基础类型, 还能直接存储对象、Parcelable、byte[]等, 无需手动序列化:

Code block

```

1 // 1. 写入数据 (直接put, 无需Editor)
2 // 基础类型
3 mmkv.putString("user_name", "张三");
4 mmkv.putInt("user_age", 25);
5 mmkv.putBoolean("is_login", true);
6 mmkv.putLong("login_time", System.currentTimeMillis());
7
8 // 特殊类型: 直接存储对象 (需实现Parcelable接口)
9 User user = new User("李四", 28, "13800138000");
10 mmkv.encode("user_obj", user); // encode()与put()功能一致, 可混用
11
12 // 特殊类型: 存储byte[] (如图片二进制数据)
13 byte[] imageBytes = getImageBytes(); // 自定义方法获取图片字节数组
14 mmkv.putBytes("avatar", imageBytes);
15
16 // 2. 读取数据
17 String userName = mmkv.getString("user_name", "未知用户");
18 int userAge = mmkv.getInt("user_age", 0);
19 boolean isLogin = mmkv.getBoolean("is_login", false);
20
21 // 读取对象 (需传入类型模板)
22 User savedUser = mmkv.decodeParcelable("user_obj", User.class);
23
24 // 读取byte[]
25 byte[] savedImageBytes = mmkv.getBytes("avatar", null);
26
27 // 3. 批量操作 (支持事务)
28 mmkv.beginTransaction(); // 开启事务
29 try {
30     mmkv.putString("key1", "value1");
31     mmkv.putInt("key2", 100);

```

```

32     mmkv.commitTransaction(); // 提交事务
33 } catch (Exception e) {
34     mmkv.rollbackTransaction(); // 异常时回滚
35 }

```

注意：存储对象时，实体类需实现 `Parcelable` 接口（MMKV也支持Gson序列化，需额外配置），示例User类：

Code block

```

1  public class User implements Parcelable {
2      public String name;
3      public int age;
4      public String phone;
5
6      // 构造方法
7      public User(String name, int age, String phone) {
8          this.name = name;
9          this.age = age;
10         this.phone = phone;
11     }
12
13     // Parcelable 接口实现 (可通过Android Studio自动生成)
14     protected User(Parcel in) {
15         name = in.readString();
16         age = in.readInt();
17         phone = in.readString();
18     }
19
20     public static final Creator<User> CREATOR = new Creator<User>() {
21         @Override
22         public User createFromParcel(Parcel in) {
23             return new User(in);
24         }
25
26         @Override
27         public User[] newArray(int size) {
28             return new User[size];
29         }
30     };
31
32     @Override
33     public int describeContents() {
34         return 0;
35     }
36

```

```

37     @Override
38     public void writeToParcel(Parcel dest, int flags) {
39         dest.writeString(name);
40         dest.writeInt(age);
41         dest.writeString(phone);
42     }
43 }

```

④ 删除数据与清空

Code block

```

1  // 删除单个key
2  mmkv.remove("user_age");
3
4  // 删除多个key
5  mmkv.remove(new String[]{"key1", "key2"});
6
7  // 清空当前实例的所有数据
8  mmkv.clear();
9
10 // 清空所有MMKV存储的数据（谨慎使用）
11 MMKV.clearAll();

```

⑤ 监听数据变化

支持监听单个key或所有key的变化，比SP的监听更灵活：

Code block

```

1  // 监听单个key的变化
2  mmkv.registerOnKeyValueChangedListener(new MMKV.OnKeyValueChangedListener() {
3      @Override
4      public void onKeyValueChanged(MMKV mmkv, String key, Object oldValue,
5      Object newValue) {
6          // key对应的数据变化时回调
7          if ("is_login".equals(key)) {
8              boolean newState = (boolean) newValue;
9              updateLoginUI(newState);
10         }
11     });
12
13 // 监听所有key的变化（全局监听）
14 MMKV.registerOnMMKVContentChangedListener(new
15     MMKV.OnMMKVContentChangedListener() {

```



```

15     @Override
16     public void onContentChanged(MMKV mmkv) {
17         // 任意MMKV实例的数据变化时回调
18         Log.d("MMKV", "存储内容发生变化, 实例ID: " + mmkv.mmapID());
19     }
20 });
21
22 // 解注册监听 (避免内存泄漏)
23 @Override
24 protected void onDestroy() {
25     super.onDestroy();
26     mmkv.unregisterOnKeyValueChangedListener(keyListener);
27     MMKV.unregisterOnMMKVContentChangedListener(globalListener);
28 }

```

(4) MMKV的优缺点及适用场景

- **优点：**性能远超SP；支持多数据类型（含对象）；API简洁；线程/进程安全；体积小（核心包仅几十KB）。
- **缺点：**需引入第三方依赖；存储对象需实现Parcelable（略繁琐）。
- **适用场景：**替代SP的所有场景，尤其适合“需要存储对象”“对性能要求高”“跨进程通信”的场景，如用户信息存储、高频读写的配置项。

2. DataStore — Google官方替代方案（推荐AndroidX项目）

DataStore是Google在Jetpack组件中推出的新一代数据持久化方案，旨在彻底替代SP，解决SP的异步提交风险、类型不安全等问题。分为 `Preferences DataStore`（键值对存储）和 `Proto DataStore`（对象存储，基于Protocol Buffers）两种。

(1) 核心特性

- **类型安全：**基于Kotlin协程和Flow，支持编译时类型检查（Java使用需配合RxJava）；
- **异步安全：**所有操作均为异步，避免ANR（无SP的apply/commit问题）；
- **支持对象存储：**Proto DataStore可直接存储复杂对象，无需手动序列化；
- **生命周期感知：**与Jetpack组件兼容，自动管理生命周期；
- **兼容性：**支持Android 6.0（API 23）及以上，需依赖AndroidX。

(2) 集成步骤

根据需求选择Preferences或Proto版本，这里以Java常用的Preferences DataStore为例：

Code block

```

1 // 模块级 build.gradle
2 dependencies {
3     // Preferences DataStore (键值对)
4     implementation "androidx.datastore:datastore-preferences:1.0.0"
5
6     // 若用Java, 需配合RxJava (DataStore原生基于Kotlin协程)
7     implementation "androidx.datastore:datastore-preferences-rxjava3:1.0.0"
8     implementation "io.reactivex.rxjava3:rxandroid:3.0.2"
9 }

```

(3) Preferences DataStore 使用流程 (Java+RxJava3)

① 创建DataStore实例 (单例模式)

DataStore建议单例使用，避免重复创建导致资源泄漏：

Code block

```

1 import androidx.datastore.core.DataStore;
2 import androidx.datastore.preferences.core.Preferences;
3 import androidx.datastore.preferences.rxjava3.RxPreferenceDataStoreBuilder;
4 import androidx.datastore.preferences.core.PreferencesKeys;
5 import androidx.datastore.preferences.core.StringPreference;
6
7 public class DataStoreManager {
8     // 定义Key (编译时类型安全, 避免key拼写错误)
9     public static final StringPreference USER_NAME =
10     PreferencesKeys.stringKey("user_name");
11     public static final Preferences.Key<Integer> USER_AGE =
12     PreferencesKeys.intKey("user_age");
13     public static final Preferences.Key<Boolean> IS_LOGIN =
14     PreferencesKeys.booleanKey("is_login");
15
16     // 单例实例
17     private static DataStoreManager instance;
18     private final DataStore<Preferences> dataStore;
19
20     private DataStoreManager(Context context) {
21         // 创建DataStore实例 (文件名: my_app_preferences)
22         dataStore = new RxPreferenceDataStoreBuilder(context,
23         "my_app_preferences").build();
24     }
25
26     public static synchronized DataStoreManager getInstance(Context context) {
27         if (instance == null) {
28             instance = new DataStoreManager(context.getApplicationContext());
29         }
30     }
31 }

```

```

26         return instance;
27     }
28
29     // 获取DataStore实例
30     public DataStore<Preferences> getDataStore() {
31         return dataStore;
32     }
33 }

```

② 写入数据（异步）

通过RxJava的 `updateData` 方法写入，自动异步执行：

Code block

```

1  // 获取DataStore实例
2  DataStore<Preferences> dataStore =
    DataStoreManager.getInstance(this).getDataStore();
3
4  // 写入数据（RxJava链式调用）
5  dataStore.updateDataAsync(prefs -> {
6      // 创建修改后的Preferences对象
7      MutablePreferences mutablePrefs = prefs.toMutablePreferences();
8      mutablePrefs.set(DataStoreManager.USER_NAME, "张三");
9      mutablePrefs.set(DataStoreManager.USER_AGE, 25);
10     mutablePrefs.set(DataStoreManager.IS_LOGIN, true);
11     // 返回修改后的结果（异步提交）
12     return Single.just(mutablePrefs);
13 })
14 .subscribeOn(Schedulers.io()) // 子线程执行
15 .observeOn(AndroidSchedulers.mainThread()) // 主线程回调
16 .subscribe(
17     unused -> {
18         // 写入成功回调
19         Toast.makeText(this, "写入成功", Toast.LENGTH_SHORT).show();
20     },
21     throwable -> {
22         // 写入失败回调
23         Log.e("DataStore", "写入失败: " + throwable.getMessage());
24     }
25 );

```

③ 读取数据（监听变化）

通过RxJava的 `data` 方法获取Flow流，可实时监听数据变化：

```

1 // 1.1.1 读取单个key的数据并监听变化
2     datastore.data()
3         .map(prefs -> {
4             // 读取数据, 指定默认值
5             String userName = prefs.get(DataStoreManager.USER_NAME) != null ?
prefs.get(DataStoreManager.USER_NAME) : "未知用户";
6             boolean isLogin = prefs.get(DataStoreManager.IS_LOGIN) != null ?
prefs.get(DataStoreManager.IS_LOGIN) : false;
7             // 可返回自定义实体类封装数据
8             return new UserInfo(userName, isLogin);
9         })
10        .subscribeOn(Schedulers.io())
11        .observeOn(AndroidSchedulers.mainThread())
12        .subscribe(
13            userInfo -> {
14                // 数据变化时回调 (首次订阅也会触发)
15                tvUserName.setText(userInfo.getName());
16                updateLoginUI(userInfo.isLogin());
17            },
18            throwable -> {
19                Log.e("DataStore", "读取失败: " + throwable.getMessage());
20            }
21        );
22
23 // 读取所有数据
24     datastore.data()
25         .subscribeOn(Schedulers.io())
26         .observeOn(AndroidSchedulers.mainThread())
27         .subscribe(
28             prefs -> {
29                 // 遍历所有key-value
30                 for (Preferences.Key<?> key : prefs.asMap().keySet()) {
31                     Object value = prefs.get(key);
32                     Log.d("DataStore", key.getName() + ": " + value);
33                 }
34             },
35             throwable -> { /* 处理异常 */ }
36        );

```

④ 删除数据

Code block

```

1 // 删除单个key
2     datastore.updateDataAsync(prefs -> {
3         MutablePreferences mutablePrefs = prefs.toMutablePreferences();

```

```

4     mutablePrefs.remove(DataStoreManager.USER_AGE); // 删除指定key
5     return Single.just(mutablePrefs);
6 }
7 .subscribe(/* 处理结果 */);
8
9 // 清空所有数据
10 datastore.updateDataAsync(prefs -> {
11     MutablePreferences mutablePrefs = prefs.toMutablePreferences();
12     mutablePrefs.clear(); // 清空所有key
13     return Single.just(mutablePrefs);
14 })
15 .subscribe(/* 处理结果 */);

```

(4) DataStore的优缺点及适用场景

- **优点：** Google官方推荐；类型安全；异步安全无ANR；支持Flow监听；Proto版本可高效存储复杂对象。
- **缺点：** Java使用需依赖RxJava，代码较繁琐；兼容性要求较高（API 23+）；首次集成成本略高。
- **适用场景：** AndroidX项目；对类型安全和异步稳定性要求高的场景；长期迭代的项目（符合Google未来技术方向）。

3. Hawk —— 简化版对象存储库

Hawk是一款基于SP、AES加密和Gson的封装库，核心优势是“一键存储对象”，API极其简洁，适合快速开发。

(1) 核心特性

- **API极简：** 一行代码实现对象存储；
- **支持加密：** 可对存储数据进行AES加密，保护敏感信息；
- **自动序列化：** 基于Gson，无需手动实现Parcelable；
- **支持数据类型：** 所有可被Gson序列化的对象（实体类、集合、Map等）。

(2) 集成步骤

Code block

```

1 // 模块级 build.gradle
2 dependencies {
3     // Hawk 核心依赖
4     implementation 'com.orhanobut:hawk:2.0.1'
5 }

```

(3) 使用流程 (Java)

① 初始化 (Application中)

Code block

```
1  public class MyApp extends Application {
2      @Override
3      public void onCreate() {
4          super.onCreate();
5          // 基础初始化
6          Hawk.init(this).build();
7
8          // 进阶初始化 (带加密)
9          Hawk.init(this)
10             .setEncryptionMethod(Hawk.EncryptionMethod.AES) // AES加密
11             .setPassword("my_app_secret_key") // 加密密钥 (自定义)
12             .setStorage(Hawk.Storage.SHARED_PREFERENCES) // 存储基于SP (默认)
13             .build();
14     }
15 }
```

② 读写数据 (一行实现)

Code block

```
1  // 1. 存储对象 (无需序列化, Gson自动处理)
2  User user = new User("张三", 25, "13800138000");
3  Hawk.put("user_obj", user); // 存储对象
4  Hawk.put("user_list", Arrays.asList(user, new User("李四", 28))); // 存储集合
5
6  // 2. 存储基础类型
7  Hawk.put("is_login", true);
8  Hawk.put("login_time", System.currentTimeMillis());
9
10 // 3. 读取数据 (指定类型)
11 User savedUser = Hawk.get("user_obj", new User()); // 第二个参数为默认值
12 List<User> userList = Hawk.get("user_list", new ArrayList<>());
13 boolean isLogin = Hawk.get("is_login", false);
14
15 // 4. 检查key是否存在
16 boolean hasUser = Hawk.contains("user_obj");
17
18 // 5. 删除数据
19 Hawk.delete("user_obj"); // 删除单个key
20 Hawk.deleteAll(); // 清空所有数据
```

(4) Hawk的优缺点及适用场景

- **优点：**API极简，开发效率高；支持加密和复杂对象；无需手动序列化。
- **缺点：**基于SP实现，大量数据性能差；依赖Gson，可能与项目中Gson版本冲突；维护频率较低（近年更新少）。
- **适用场景：**小型项目；快速原型开发；需要存储对象但不想处理序列化的场景。

4. SecurePreferences —— 加密版SP（敏感数据专用）

SecurePreferences是SP的加密增强版，核心功能是对存储的key和value进行AES加密，解决SP数据明文存储的安全问题，适合存储密码、Token等敏感信息。

(1) 核心特性

- **全程加密：**key和value均通过AES加密，存储在XML文件中为密文；
- **API兼容：**用法与原生SP完全一致，无需学习新API；
- **安全可靠：**支持自定义密钥生成方式，避免密钥硬编码风险。

(2) 集成步骤

Code block

```
1 // 模块级 build.gradle
2 dependencies {
3     implementation 'com.scottyab:secure-preferences-lib:0.1.7'
4 }
```

(3) 使用流程（Java）

API与原生SP完全一致，仅初始化方式不同：

Code block

```
1 // 1. 初始化（获取加密SP实例）
2 // 方式1：基础加密（密钥通过设备信息生成，无需手动传入）
3 SharedPreferences secureSP = new SecurePreferences(this, "my_password",
4     "secure_sp.xml");
5 // 方式2：自定义密钥（推荐，提升安全性）
6 String secretKey = generateSecureKey(); // 自定义方法生成密钥（如从服务端获取）
7 SharedPreferences secureSP = new SecurePreferences(this, secretKey,
8     "secure_sp.xml");
9 // 2. 读写数据（与原生SP完全一致）
```

```
10  SharedPreferences.Editor editor = secureSP.edit();
11  editor.putString("user_token", "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...") // 存
    储Token（加密）
12      .putString("pay_password", "123456") // 存储敏感信息（加密）
13      .apply();
14
15  // 读取数据（自动解密）
16  String token = secureSP.getString("user_token", "");
17  String payPwd = secureSP.getString("pay_password", "");
18
19  // 其他操作（删除、监听）与原生SP一致
```

(4) SecurePreferences的优缺点及适用场景

- **优点：**API与SP兼容，迁移成本低；加密彻底，保护敏感数据；轻量级无额外依赖。
- **缺点：**仅解决加密问题，未解决SP的性能和数据类型问题；密钥管理需谨慎（避免硬编码）。
- **适用场景：**需要存储敏感信息（Token、密码、身份证号）的场景；对SP有依赖但需加密的旧项目。

三、存储方案选型建议

结合前文内容，不同场景下的最优选择如下表所示，可根据项目需求快速决策：

场景需求	推荐方案	核心理由
简单配置存储（少量键值对，无敏感信息）	原生SharedPreferences	无依赖、上手快、满足基础需求
需要存储对象/集合，追求高性能	MMKV	性能强、API简洁、支持多类型、腾讯背书
AndroidX项目，长期迭代，重视类型安全	DataStore（Proto版本）	官方推荐、异步安全、符合未来技术方向
小型项目/原型开发，快速实现对象存储	Hawk	API极简、无需序列化、开发效率高
存储敏感信息（Token、密码）	SecurePreferences / MMKV（带加密）	加密彻底，前者兼容SP，后者性能更优
跨进程数据共享	MMKV（开启跨进程模式）	原生支持、性能优于AIDL+SP

💡 总结：原生SP适合入门和简单场景；MMKV是当前综合最优的替代方案，推荐大多数项目使用；DataStore适合AndroidX新项目的长期规划；敏感数据优先考虑加密方案。

四、常见问题解答

1. SP的apply()方法会导致ANR吗？

不会。apply()是异步提交，将修改写入内存后立即返回，后台线程异步写入磁盘；而commit()是同步提交，会阻塞当前线程（如主线程调用可能导致ANR）。建议优先使用apply()，重要数据可使用commit()并处理返回结果。

2. MMKV存储的对象必须实现Parcelable吗？

不是。MMKV支持多种序列化方式：除了Parcelable，还可通过Gson、FastJson等工具将对象转为String存储，或使用MMKV的 `encode(String key, Object object)` 方法（内部通过Gson序列化，需额外配置）。

3. DataStore与SP如何迁移？

Google提供了SP到DataStore的迁移工具，可通过 `RxSharedPreferencesMigration` 类将SP数据批量迁移到DataStore，确保数据不丢失，具体可参考Google官方迁移文档。

4. 敏感数据存储除了加密库，还有其他方案吗？

有。对于高度敏感的数据（如支付密码、私钥），推荐使用Android系统的 `KeyStore` 结合加密库使用：将加密密钥存储在KeyStore中（硬件级安全），再用密钥对数据加密存储，避免密钥泄露风险。

（注：文档部分内容可能由 AI 生成）